EXPERIMENT NO: 1

Environment: Microsoft Windows/Linux

Objective: Implement all basic commands used in Linux OS

a. ls	**e. man**	**i. rmdir**
b. mkdir	**f. date**	**j. head**
c. cd	**g. cal**	**k. tail**
d. cat	**h. rm**	**l. pwd**

Procedure/Algorithm/Steps/Syntax:

The commands used in UNIX/LINUX are:

S. No.	Command	Example	Implementation
a.	**ls**	ls	Lists files in current directory
		ls -a	List all files in current directory
b.	**mkdir**	mkdir graphics	Make a directory called graphics
c.	**cd**	cd tempdir	Change directory to tempdir
		cd ..	Move back one directory
		cd ~gla/student	Move into gla's student directory
d.	**cat**	cat filename	Opens an existing file
e.	**man**	man ls	Online manual (help) about command
f.	**date**	date	Print out current date
g.	**cal <mo> <yr>**	cal 9 2000	Print calendar for September 2000
h.	**rm**	rm file1.bak	Remove or delete file
		rm *.tmp	Remove all file with .tmp extension
i.	**rmdir**	rmdir emptydir	Remove directory (must be empty)
j.	**head**	head <filename>	Displays first 10 lines of the file
k.	**tail**	tail <filename>	Displays last 10 lines of the file
l.	**pwd**	pwd	Prints current working directory

Code:

As according to the table given above

Sample Input:

As according to the table given above

Sample Output:

As according to the table given above

Post Experiment Question:

1. Is Linux GUI based or CUI based?
2. Is UNIX GUI based or CUI based?
3. What is the use of ls, cat, head and tail?
4. What are read write execute permissions on a directory?
5. What is process based commands?

EXPERIMENT NO: 2

Environment: Microsoft Windows/Linux

Objective: Implement all basic commands used in Linux OS

a. cp	**e. who**	**i. bc**
b. mv	**f. whoami**	**j. top**
c. sort	**g. ps**	**k. grep**
d. cut	**h. kill**	**l. chmod**

Procedure/Algorithm/Steps/Syntax:

The commands used in UNIX/LINUX are:

S. No.	Command	Example	Implementation
a.	**cp**	cp file1 file2	Copy file into directory
		cp file1 file1.bak	Make backup of file1
b.	**mv**	mv file1 file2	Move or rename files
c.	**sort**	sort file1	Sorts the contents of the file alphabetically
		sort –n file1	Sorts the contents of the file numerically
		sort –r file1	Sorts the contents of the file alphabetically in reverse order
d.	**cut**	cut –m -n filename	Displays only the columns from m to n of the file
e.	**who**	who	Lists who is logged on your machine
f.	**whoami**	whoami	Lists the username of the current user logged in.
g.	**ps <opt>**	ps aux	List all running processes by #ID
		ps aux | grep dhyatt	List process #ID's running by dhyatt
h.	**kill <opt> <ID>**	kill -9 8453	Kill process with ID #8453
i.	**bc**	bc	starts a calculator
j.	**top**	top	Various fields are displayed as per the current scenario
k.	**grep <str><files>**	grep "bad word" file1	Find which lines in a file contain a certain word
l.	**chmod <opt> <file>**	chmod 644 *.html	Change file permissions read only
		chmod 755 file.exe	Change file permissions to executable

Options with ps command

Various options for ps are:

Option	Description
-a	Displays all processes on a terminal, with the exception of group leaders.
-c	Displays scheduler data.
-d	Displays all processes with the exception of session leaders.
-e	Displays all processes.
-f	Displays a full listing.
-g*list*	Displays data for the *list* of group leader IDs.
-j	Displays the process group ID and session ID.
-l	Displays a long listing
-p*list*	Displays data for the *list* of process IDs.
-s*list*	Displays data for the *list* of session leader IDs.
-t*list*	Displays data for the *list* of terminals.
-u*list*	Displays data for the *list* of usernames.

Details of top command

On implementing top command, the following fields are displayed:

1. CODE -- Code Size (KB).
2. COMMAND -- Command Name or Command Line.
3. %CPU -- CPU Usage.
4. DATA -- Data + Stack Size (KB).
5. Flags -- Task Flags.
6. GID -- Group Id.
7. GROUP -- Group Name.
8. %MEM -- Memory Usage
9. NI -- Nice Value.
10. nDRT -- Dirty Pages Count.
11. nMaj -- Major Page Fault Count.
12. nMin -- Minor Page Fault count.
13. nTH -- Number of Threads.
14. PID -- Process Id.
15. PPID -- Parent Process Id.
16. RES -- Resident Memory Size (KB), etc.

Code:

As according to the table given above

Sample Input:

As according to the table given above

Sample Output:

As according to the table given above

Post Experiment Question:

1. Explain the syntax of ps with any 2 options.
2. What are the various options of sort command?
3. What are the various options of bc command?
4. What is the meaning of obase and ibase?
5. What is the significance of scale?
6. What is the significance of grep command?
7. What are the various options used in grep command?

EXPERIMENT NO: 3

Environment: Microsoft Windows/Linux

Objective: Implement all basic commands used in Linux OS

a. nohup	e. pg	i. sed
b. last	f. history	j. emacs
c. more	g. touch	k. du
d. less	h. tr	l. diff

Procedure/Algorithm/Steps/Syntax:
The commands used in UNIX/LINUX are:

S. No.	Command	Example	Implementation
a.	**nohup**	nohup find -size +100k > log.txt	Runs a command, making it immune to any HUP (hangup) signals while it is running. Runs the find command, instructing it to search for any file bigger then 100k. Find will continue to search regardless of the user's connection to the terminal, and log results to the file log.txt
b.	**last**	last	Finds out who has recently logged in and out on your server
c.	**more**	more file1	Look at file, one page at a time
d.	**less**	less file1	Less is a program similar to more, but which allows backward movement in the file as well as forward movement.
e.	**pg**	pg <filename>	pg displays a text file on a CRT one screenful at once. After each page, a prompt is displayed. The user may then either press the newline key to view the next page
f.	**history**	history	Lists commands you've done recently
g.	**touch**	touch <filename>	Creates a new file of zero bytes, if does not exist. If, however exists, changes the timestamp of it.
h.	**tr**	tr [a-z] [A-Z] <filename>	Translates in the file as per given translation. In the example, changes the lowercase characters to uppercase characters.
i.	**sed**	sed 's/unix/linux/' file.txt	Sed command is mostly used to replace the text in a file. The

			below simple sed command replaces the word "unix" with "linux" in the file.
j.	**emacs**	emacs filename	emacs is a screen editor.
k.	**du**	du <filename>	Lists the disk usage of the given file.
l.	**diff**	diff <file1> <file2>	Checks whether two files are same or different.

Code:
As according to the table given above

Sample Input:
As according to the table given above

Sample Output:
As according to the table given above

Post Experiment Question:

1. Explain the syntax of nohup.
2. How does diff command work?
3. What does history command give as output?
4. What is the difference between tr and sed?
5. Where does the output of the command given with nohup gets stored by default?

EXPERIMENT NO: 4

Environment: Microsoft Windows/Linux

Objective: Write and implement the basic vi editor commands.

Procedure/Algorithm/Steps/Syntax:
vi is a visual text editor.
vi is actually the command which launches the visual mode of ex. As ex gained popularity, he noticed that most users were exclusively using its visual mode, so he created a link to ex which immediately starts in visual mode and simply named it vi. The name stuck, and today vi is the most popular text editor among Linux users.

Code:
vi [option]

-l	Set up for editing LISP programs.
-R	Readonly mode; the readonly flag is set, preventing accidental overwriting of the file.
-r filename	Edit filename after an editor or system crash. (Recovers the version of filename that was in the buffer when the crash occurred.)
-S	This option is used in conjunction with the -t tag option to tell vithat the tags file may not be sorted and that, if the binary search (which relies on a sorted tags file) for tag fails to find it, the much slower linear search should also be done. Since the linear search is slow, users of large tags files should ensure that the tags files are sorted rather than use this flag. Creation of tags files normally produces sorted tags files. See ctags for more information on tags files.
-t tag	Edit the file containing the tag tag, and position the editor at its definition.
-v	Start up in display editing state using vi. You can achieve the same effect by typing the vi command itself.
-V	Verbose mode. When ex commands are read by means ofstandard input, the input will be echoed to standard error. This may be useful when processing ex commands within shell scripts.
-x	Encryption option; when used, vi simulates the X command of exand prompts the user for a key. This key is used to encrypt and decrypt text using the algorithm of the crypt command.
-wn	Set the default window size to n. This is useful when using the editor over a slow speed line.

-C	Encryption option; same as the -x option, except that visimulates the C command of ex. The C command is like the Xcommand of ex, except that all text read in is assumed to have been encrypted.
+command | -c command	Begin editing by executing the specified editor command (usually a search or positioning command).
filename	The file to be edited.

Sample Input:

vi myfile.txt

Sample Output:
Edits the file myfile.txt

Post Experiment Question:

1. Why is vi so popular?
2. What are the various modes of vi editor?